

TRABAJO PRÁCTICO 8: INTERFACES Y EXCEPCIONES EN JAVA



Resolución Parte 1: Interfaces en un sistema de E-commerce:

1.1 Creación del proyecto

Se creó un nuevo proyecto Java en **NetBeans** llamado **TP8_InterfacesExcepciones**.

Dentro del proyecto se agregaron las clases e interfaces solicitadas en el trabajo práctico.

Se utilizó el paquete tp8 para mantener una estructura ordenada.

1.2 Interfaz Pagable

Descripción:

La interfaz Pagable define el contrato para todos los elementos que puedan calcular un monto total. Cualquier clase que implemente esta interfaz debe proporcionar su propia versión del método calcularTotal().

Código:

```
public interface Pagable {  
    double calcularTotal();  
}
```

1.3 Clase Producto

Descripción:

La clase Producto representa un ítem del sistema de e-commerce. Implementa la interfaz Pagable y devuelve su precio como total. Incluye atributos nombre y precio, un constructor, y métodos *getter*.

Código:

```
public class Producto implements Pagable {  
    private String nombre;  
    private double precio;  
  
    public Producto(String nombre, double precio) {  
        this.nombre = nombre;  
        this.precio = precio;  
    }  
}
```

```

    public String getNombre() { return nombre; }
    public double getPrecio() { return precio; }

    @Override
    public double calcularTotal() {
        return precio;
    }

    @Override
    public String toString() {
        return nombre + " ($" + precio + ")";
    }
}

```

1.4 Clase Pedido

Descripción:

La clase Pedido agrupa varios productos y mantiene un estado. Implementa Pagable para calcular el total del pedido y utiliza la interfaz Notificable para informar al cliente los cambios de estado.

Código:

```

import java.util.ArrayList;
import java.util.List;

public class Pedido implements Pagable {
    public enum Estado { CREADO, PAGADO, ENVIADO, ENTREGADO }

    private List<Producto> productos;
    private Estado estado;
    private Notificable clienteNotificable;

    public Pedido() {
        this.productos = new ArrayList<>();
        this.estado = Estado.CREADO;
    }
}

```

```

public void agregarProducto(Producto p) {
    productos.add(p);
}

public void removerProducto(Producto p) {
    productos.remove(p);
}

@Override
public double calcularTotal() {
    double total = 0;
    for (Producto p : productos) total += p.calcularTotal();
    return total;
}

public void setClienteNotificable(Notificable n) {
    this.clienteNotificable = n;
}

public void cambiarEstado(Estado nuevoEstado) {
    this.estado = nuevoEstado;
    if (clienteNotificable != null) {
        clienteNotificable.notificar("El pedido cambió a: " + nuevoEstado);
    }
}

@Override
public String toString() {
    return "Pedido [estado=" + estado + ", total=$" + calcularTotal() + "]";
}
}

```

1.5 Interfaces de pago (Pago, PagoConDescuento)

Descripción:

Estas interfaces definen el comportamiento de los distintos medios de pago.

Pago exige implementar el método `procesarPago()`, mientras que `PagoConDescuento` añade `aplicarDescuento()` para calcular rebajas sobre el monto total.

Código:

```
public interface Pago {  
    boolean procesarPago(double monto);  
}  
  
public interface PagoConDescuento extends Pago {  
    double aplicarDescuento(double monto, double porcentaje);  
}
```

1.6 Clases `TarjetaCredito` y `PayPal`

Descripción:

`TarjetaCredito` implementa `PagoConDescuento`, aplicando descuentos y simulando el procesamiento del pago. `PayPal` implementa `Pago` sin descuentos. Ambas imprimen mensajes simulando el cobro.

Código:

```
public class TarjetaCredito implements PagoConDescuento {  
    private String numero;  
    private String titular;  
  
    public TarjetaCredito(String numero, String titular) {  
        this.numero = numero;  
        this.titular = titular;  
    }  
  
    @Override  
    public double aplicarDescuento(double monto, double porcentaje) {  
        return monto * (1 - porcentaje / 100.0);  
    }  
  
    @Override  
    public boolean procesarPago(double monto) {  
        System.out.println("Procesando pago con Tarjeta de " + titular + " por $" + monto);
```

```

        return true;
    }
}

public class PayPal implements Pago {
    private String cuentaEmail;

    public PayPal(String cuentaEmail) {
        this.cuentaEmail = cuentaEmail;
    }

    @Override
    public boolean procesarPago(double monto) {
        System.out.println("Procesando pago con PayPal cuenta " + cuentaEmail + " por $" +
monto);
        return true;
    }
}

```

1.7 Interfaz Notificable y clase Cliente

Descripción:

Cliente implementa Notificable, permitiendo recibir mensajes cuando cambia el estado del pedido. Esto muestra cómo se logra el desacoplamiento mediante interfaces.

Código:

```

public interface Notificable {
    void notificar(String mensaje);
}

public class Cliente implements Notificable {
    private String nombre;
    private String email;

    public Cliente(String nombre, String email) {
        this.nombre = nombre;
        this.email = email;
    }
}

```

```

@Override

public void notificar(String mensaje) {
    System.out.println("Notificación para " + nombre + " (" + email + "): " + mensaje);
}

@Override

public String toString() {
    return nombre + " <" + email + ">";
}
}

```

1.8 Clase EcommerceDemo (Programa principal)

Descripción:

Esta clase integra todos los componentes del sistema. Se crean objetos Cliente, Producto, Pedido y TarjetaCredito. Se muestran los pasos del proceso: creación del pedido, aplicación de descuento, pago y cambio de estado.

Código:

```

public class EcommerceDemo {
    public static void main(String[] args) {
        Cliente cliente = new Cliente("Juan Perez", "juan@ejemplo.com");
        Producto p1 = new Producto("Auriculares", 120.0);
        Producto p2 = new Producto("Mouse", 50.0);

        Pedido pedido = new Pedido();
        pedido.setClienteNotificable(cliente);
        pedido.agregarProducto(p1);
        pedido.agregarProducto(p2);

        System.out.println("Pedido creado: " + pedido);
        pedido.cambiarEstado(Pedido.Estado.PAGADO);

        TarjetaCredito tarjeta = new TarjetaCredito("1234-5678-9012-3456", "Juan Perez");
        double total = pedido.calcularTotal();
        double totalConDesc = tarjeta.aplicarDescuento(total, 10);
        if (tarjeta.procesarPago(totalConDesc)) {

```

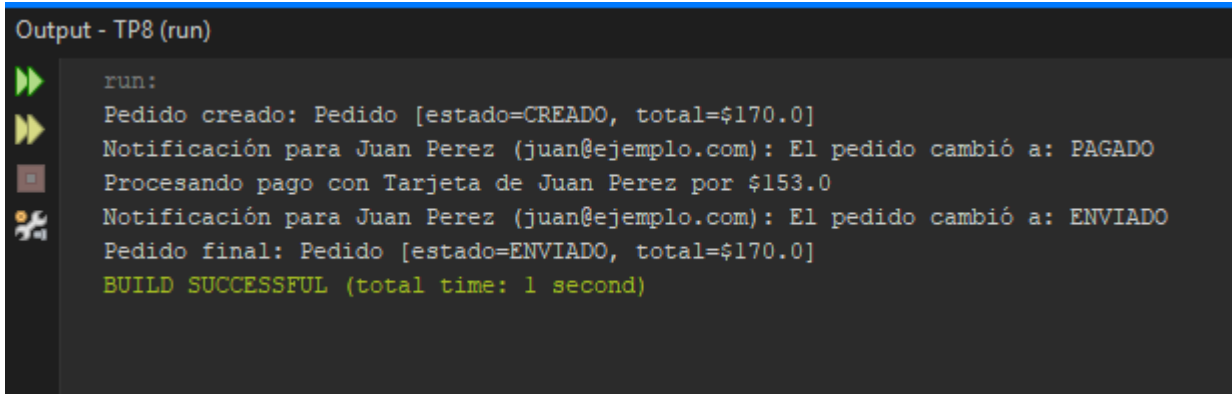
```

        pedido.cambiarEstado(Pedido.Estado.ENVIADO);
    }

    System.out.println("Pedido final: " + pedido);
}
}

```

Captura de la ejecución de la clase EcommerceDemo:



```

Output - TP8 (run)

run:
Pedido creado: Pedido [estado=CREADO, total=$170.0]
Notificación para Juan Perez (juan@ejemplo.com): El pedido cambió a: PAGADO
Procesando pago con Tarjeta de Juan Perez por $153.0
Notificación para Juan Perez (juan@ejemplo.com): El pedido cambió a: ENVIADO
Pedido final: Pedido [estado=ENVIADO, total=$170.0]
BUILD SUCCESSFUL (total time: 1 second)

```

Resolución Parte 2: Ejercicios sobre Excepciones:

2.1 División segura

Descripción:

Se manejan posibles errores al dividir números: entrada no válida y división por cero.

Código:

```

import java.util.Scanner;

public class DivisionSegura {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Dividendo: ");
            double a = Double.parseDouble(sc.nextLine());
            System.out.print("Divisor: ");
            double b = Double.parseDouble(sc.nextLine());

```

```

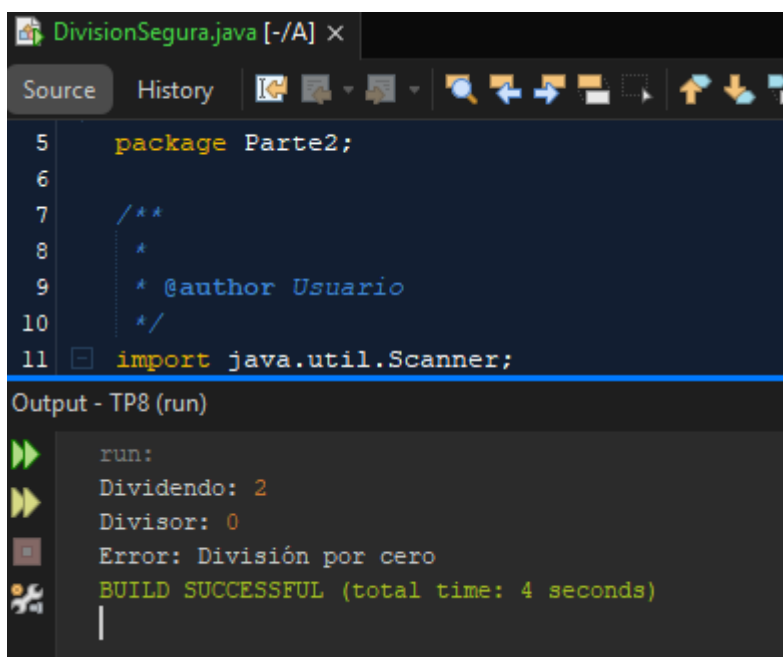
        if (b == 0) throw new ArithmeticException("División por cero");

        double resultado = a / b;

        System.out.println("Resultado: " + resultado);
    } catch (ArithmeticException | NumberFormatException e) {
        System.out.println("Error: " + e.getMessage());
    } finally {
        sc.close();
    }
}
}

```

Captura de la ejecución de DivisionSegura:



2.2 Conversión segura

Descripción:

Se utiliza try-catch para capturar errores al convertir una cadena a número entero.

Código:

```

import java.util.Scanner;

public class ConversionSegura {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Ingrese un entero: ");
    }
}

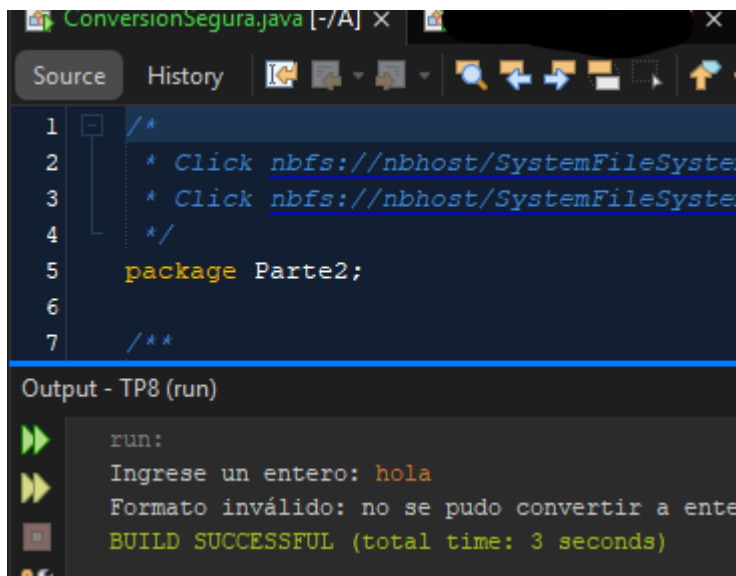
```

```

String linea = sc.nextLine();
try {
    int valor = Integer.parseInt(linea);
    System.out.println("Valor convertido: " + valor);
} catch (NumberFormatException e) {
    System.out.println("Formato inválido: no se pudo convertir a entero.");
} finally {
    sc.close();
}
}
}

```

Captura de la ejecución de ConversionSegura:



2.3 Lectura de archivo

Descripción:

Se maneja la excepción `FileNotFoundException` al intentar abrir un archivo inexistente.

Código:

```

import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class LeerArchivo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
    }
}

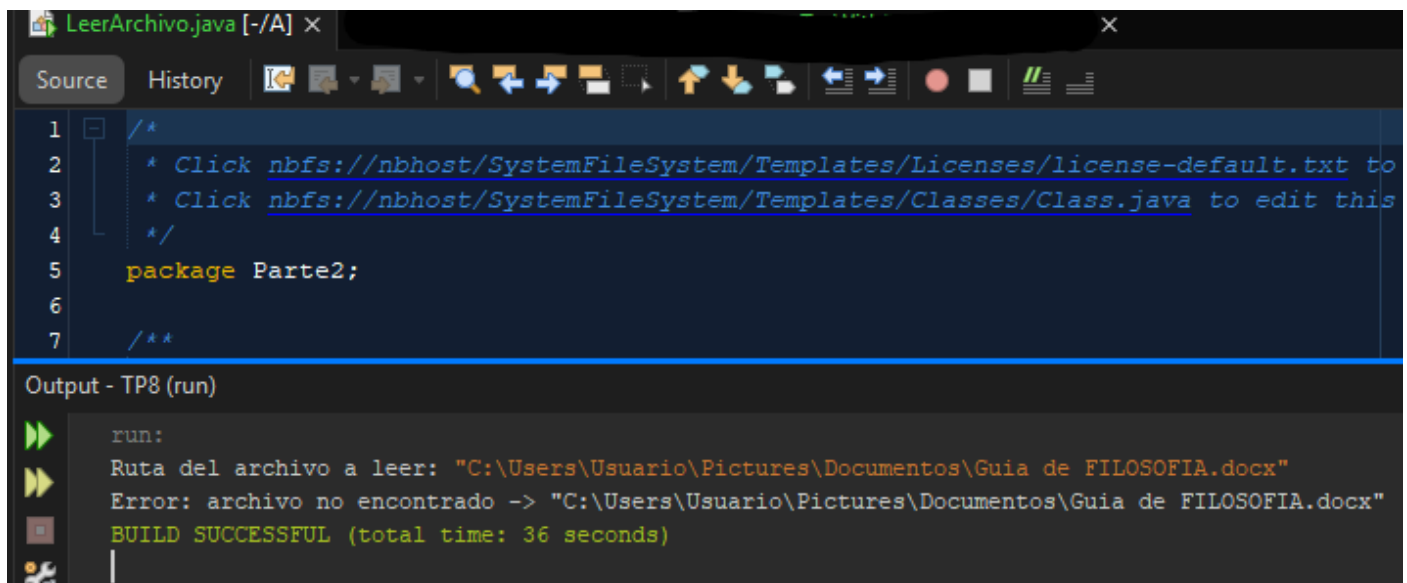
```

```

System.out.print("Ruta del archivo a leer: ");
String ruta = sc.nextLine();
try {
    File file = new File(ruta);
    Scanner fileScanner = new Scanner(file);
    while (fileScanner.hasNextLine()) {
        System.out.println(fileScanner.nextLine());
    }
    fileScanner.close();
} catch (FileNotFoundException e) {
    System.out.println("Error: archivo no encontrado -> " + ruta);
} finally {
    sc.close();
}
}
}

```

Captura de la ejecución de LeerArchivo:



2.4 Excepción personalizada EdadInvalidaException

Descripción:

Se implementó una excepción personalizada para validar la edad ingresada.

El método validarEdad() lanza la excepción si la edad no está dentro del rango permitido.

Código:

```

public class EdadInvalidaException extends Exception {
    public EdadInvalidaException(String mensaje) {
        super(mensaje);
    }
}

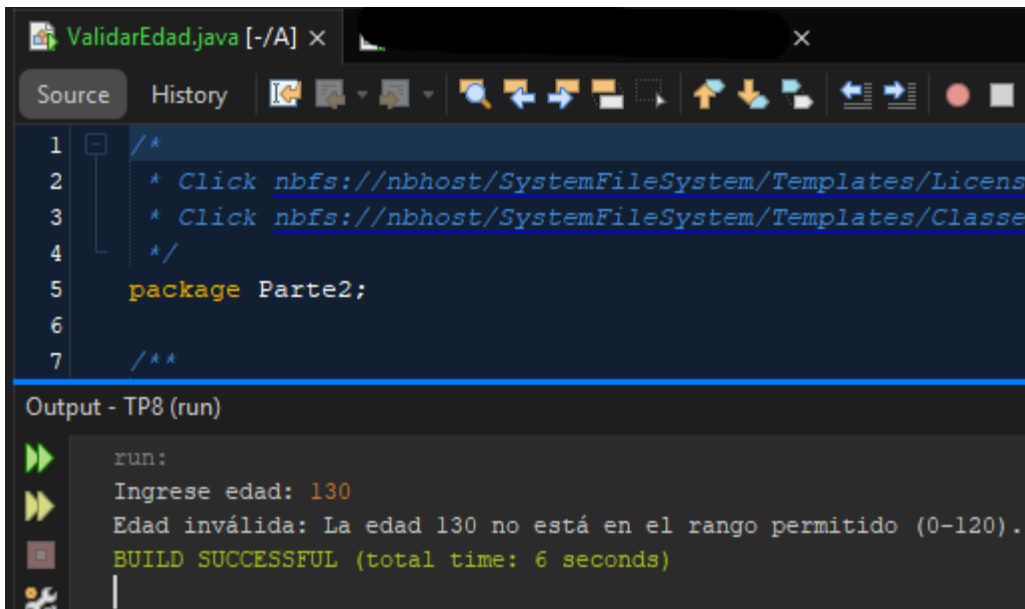
import java.util.Scanner;

public class ValidarEdad {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Ingrese edad: ");
        int edad = Integer.parseInt(sc.nextLine());
        try {
            validarEdad(edad);
            System.out.println("Edad válida: " + edad);
        } catch (EdadInvalidaException e) {
            System.out.println("Edad inválida: " + e.getMessage());
        } finally {
            sc.close();
        }
    }

    public static void validarEdad(int edad) throws EdadInvalidaException {
        if (edad < 0 || edad > 120) {
            throw new EdadInvalidaException("La edad " + edad + " no está en el rango permitido (0-120).");
        }
    }
}

```

Captura de la ejecución de ValidarEdad:



```
ValidarEdad.java [-/A] x
Source History
1 /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes
4  */
5 package Parte2;
6
7 /**
Output - TP8 (run)
run:
Ingrese edad: 130
Edad inválida: La edad 130 no está en el rango permitido (0-120).
BUILD SUCCESSFUL (total time: 6 seconds)
```

2.5 Lectura de archivo con try-with-resources

Descripción:

Se empleó el bloque try-with-resources para asegurar el cierre automático del archivo.

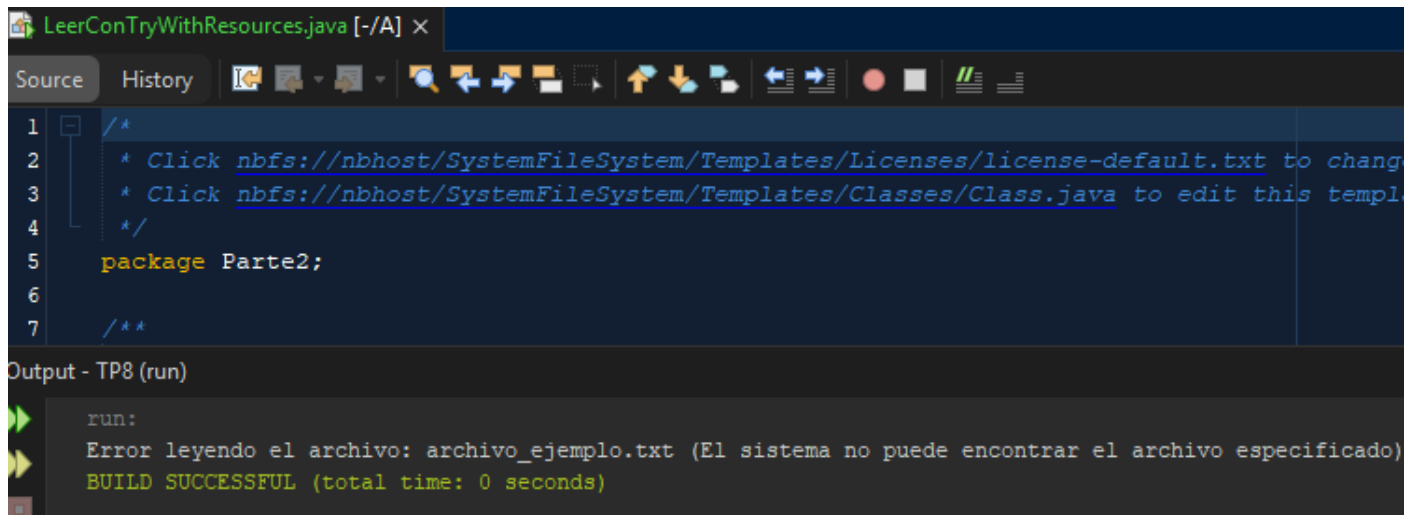
Se maneja la excepción IOException en caso de error de lectura.

Código:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class LeerConTryWithResources {
    public static void main(String[] args) {
        String ruta = "archivo_ejemplo.txt";
        try (BufferedReader br = new BufferedReader(new FileReader(ruta))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            System.out.println("Error leyendo el archivo: " + e.getMessage());
        }
    }
}
```

Captura de la ejecución de LeerConTryWithResources:



The screenshot shows an IDE window titled 'LeerConTryWithResources.java [-/A] x'. The 'Source' tab is active, displaying a Java file with the following content:

```
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to chang
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this templ
4   */
5  package Parte2;
6
7  /**
```

The 'Output - TP8 (run)' tab is also visible, showing the following output:

```
run:
Error leyendo el archivo: archivo_ejemplo.txt (El sistema no puede encontrar el archivo especificado)
BUILD SUCCESSFUL (total time: 0 seconds)
```

Link a mi repositorio de GitHub:

<https://github.com/NazarenoAranda/UTN-TUPaD-P2-Nazareno-Aranda/tree/master/UTN-TUPaD-P2-Nazareno-Aranda>

Nazareno Aranda